

Activity 5 - Organise a crate into modules

AI Vibe Coding with Rust / TGS-2023039924 / version 3.1 / K2/A2

Specification

money::format_cents(u64) -> String renders "\$D.CC". report::report_line(name, qty, unit_cents) -> Option<String> renders "<name> x<qty> @ <unit> = <total>"; report::report joins lines with "\n" and fails wholly on overflow.

Part 1: Learn Rust prompts

Explain it simply

I am learning Rust. Explain modules, pub visibility, paths with crate::, self:: and super::, and use ... as to a beginner in plain English. Use a short example from a small shop's stock system, keep the code under 20 lines, show its output, and point out one mistake beginners often make.

Walk me through the sample

Here is samples/01_inline_modules.rs from my course. Walk through it line by line. Before I run it, ask me to predict the output. Then suggest one small change I can make to experiment, and tell me what should happen.

Quiz me

Quiz me on modules, pub visibility, paths with crate::, self:: and super::, and use ... as with 5 questions, one at a time. Wait for my answer each time and explain why it is right or wrong. Include one question where I predict the output or the compiler error of a short Rust snippet.

Compare with what I know

I know some Python or JavaScript. In a small table, compare how Rust handles modules and code organisation with Python and JavaScript. Then show one Rust example and explain what the Rust compiler checks for me that the others do not.

Part 2: Build with AI (vibe-coding prompts)

Plan before code

Act as a senior Rust reviewer. Specification: money::format_cents(u64) -> String renders "\$D.CC". report::report_line(name, qty, unit_cents) -> Option<String> renders "<name> x<qty> @ <unit> = <total>"; report::report joins lines with "\n" and fails wholly on overflow. Before writing any code, list the public signatures, the edge cases and at least four boundary tests you would expect. Ask me about anything ambiguous.

Generate with AI

Implement "Organise a crate into modules" in starter/src so that the supplied tests pass. Keep the two-module layout and public paths; do not merge modules to make tests pass. Keep public signatures unchanged, use no unsafe code, and explain each design decision in one sentence.

Repair from evidence

Here is my exact cargo output: <paste the first error or failing test>. Explain the cause in plain English, propose the smallest fix that keeps the specification, and add one regression test. Do not change expected values in existing tests.

Review checklist

Signatures unchanged; no unsafe; no unwrap() on input; no new crates; every boundary tested; cargo test, clippy -D warnings and fmt --check pass; you can explain every line.

Part 3: Rust sample scripts (samples/ folder)

01_inline_modules.rs

Run: rustc --edition 2021 01_inline_modules.rs && ./01_inline_modules

// Sample: modules and privacy in one file.

```
mod money {
    pub fn format_cents(cents: u64) -> String {
        format!("{}", cents / 100, cents % 100)
    }

    fn secret_rule() -> &'static str {
        "private: only code inside `money` can call this"
    }

    pub fn explain() -> &'static str {
        secret_rule()
    }
}

mod report {
    use crate::money::format_cents; // path from the crate root

    pub fn line(name: &str, qty: u32, unit: u64) -> String {
        let total = unit * u64::from(qty);
        format!(
            "{} x{} @ {} = {}",
            name, qty, format_cents(unit),
            format_cents(total)
        )
    }
}

fn main() {
    println!("{}", report::line("Cable", 3, 250));
    println!("{}", money::explain());
    // money::secret_rule(); // error[E0603]: function `secret_rule` is private
}
```

Expected output:

Cable x3 @ \$2.50 = \$7.50
private: only code inside `money` can call this

02_nested_paths.rs

Run: rustc --edition 2021 02_nested_paths.rs && ./02_nested_paths

// Sample: nested modules, super::, self:: and use ... as.

```
mod stockpilot {
    pub mod inventory {
        pub fn count() -> u32 {
            super::defaults::STARTING_STOCK + 2
        }
    }

    mod defaults {
        pub const STARTING_STOCK: u32 = 10;
    }

    pub fn summary() -> String {
```

```

        format!("stock count = {}", self::inventory::count())
    }
}

use stockpilot::inventory::count as stock_count;

fn main() {
    println!("{}", stockpilot::summary());
    println!("via use-as: {}", stock_count());
}

```

Expected output:

```

stock count = 12
via use-as: 12

```

Learn more

Rust Book: Defining Modules to Control Scope and Privacy:

<https://doc.rust-lang.org/book/ch07-02-defining-modules-to-control-scope-and-privacy.html>

Programiz: Cargo (packages and crates): <https://www.programiz.com/rust/cargo>

Rust official learning hub (the Book, Rustlings, Rust by Example): <https://rust-lang.org/learn/>

W3Schools Rust tutorial: <https://www.w3schools.com/rust/>

W3Schools: Get Started with Rust: https://www.w3schools.com/rust/rust_getstarted.php